

Deep Q-Networks (DQN)

Q-learning for discrete actions, off policy

Playing Atari with Deep Reinforcement Learning

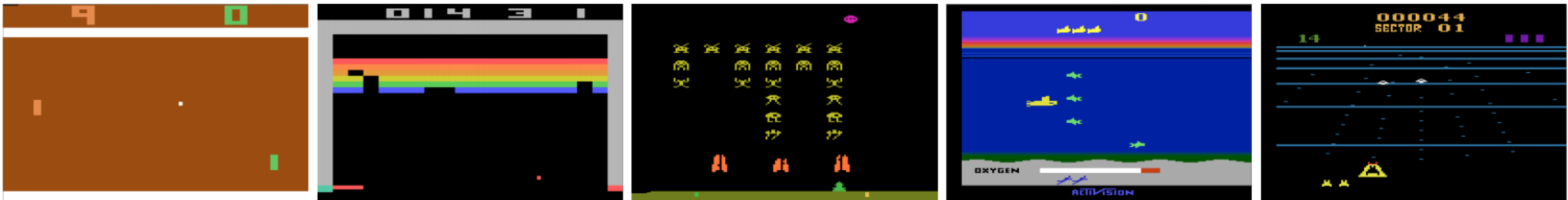
[NeurIPS 2013]

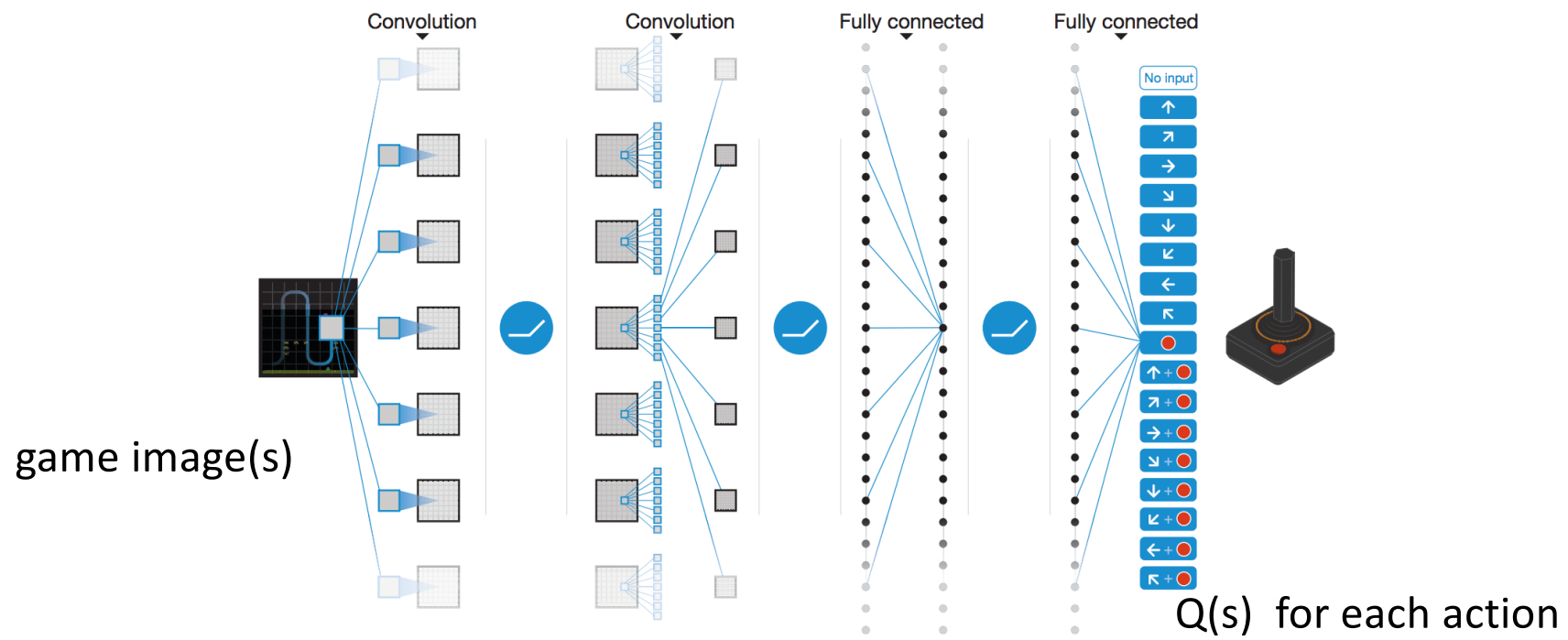
Human-level control through deep reinforcement learning

[Nature 2015]

Atari: 128 bytes RAM, 2-4 kB ROM

Atari Learning Env: [Bellemare et al, 2013]



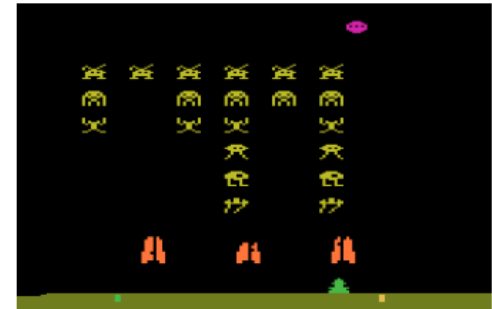


[awjuliani]

state

$x_t : 210 \times 160, 7 \text{ bits per pixel}$

$\phi(x_t) : 84 \times 84 \times 4 \text{ frames, Ychannel}$



actions

4 to 18 actions, depending on the game



DQN Algorithm

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

$\phi(s_t)$ the state for the RL problem

s_t the complete sequence at time t

sample tuples from an
“experience replay” buffer

minimize Bellman error

maintain a separate
target network, with
delayed updates

[Mnih et al., 2015]



[Nature 2015]

Preprocessing. Working directly with raw Atari 2600 frames, which are 210×160 pixel images with a 128-colour palette, can be demanding in terms of computation and memory requirements. We apply a basic preprocessing step aimed at reducing the input dimensionality and dealing with some artefacts of the Atari 2600 emulator. First, to encode a single frame we take the maximum value for each pixel colour value over the frame being encoded and the previous frame. This was necessary to remove flickering that is present in games where some objects appear only in even frames while other objects appear only in odd frames, an artefact caused by the limited number of sprites Atari 2600 can display at once. Second, we then extract the Y channel, also known as luminance, from the RGB frame and rescale it to 84×84 . The function ϕ from algorithm 1 described below applies this preprocessing to the m most recent frames and stacks them to produce the input to the Q-function, in which $m = 4$, although the algorithm is robust to different values of m (for example, 3 or 5).

As the scale of scores varies greatly from game to game, we clipped all positive rewards at 1 and all negative rewards at -1 , leaving 0 rewards unchanged. Clipping the rewards in this manner limits the scale of the error derivatives and makes it easier to use the same learning rate across multiple games. At the same time,

In these experiments, we used the RMSProp (see http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf) algorithm with minibatches of size 32. The behaviour policy during training was ϵ -greedy with ϵ annealed linearly from 1.0 to 0.1 over the first million frames, and fixed at 0.1 thereafter. We trained for a total of 50 million frames (that is, around 38 days of game experience in total) and used a replay memory of 1 million most recent frames.

In general the environment may be stochastic.

network. We refer to a neural network function approximator with weights θ as a Q-network. A Q-network can be trained by adjusting the parameters θ_i at iteration i to reduce the mean-squared error in the Bellman equation, where the optimal target values $r + \gamma \max_{a'} Q^*(s', a')$ are substituted with approximate target values $y = r + \gamma \max_{a'} Q(s', a'; \theta_i^-)$, using parameters θ_i^- from some previous iteration. This leads to a sequence of loss functions $L_i(\theta_i)$ that changes at each iteration i ,

$$\begin{aligned} L_i(\theta_i) &= \mathbb{E}_{s,a,r} [(\mathbb{E}_{s'} [y|s,a] - Q(s,a; \theta_i))^2] \\ &= \mathbb{E}_{s,a,r,s'} [(y - Q(s,a; \theta_i))^2] + \mathbb{E}_{s,a,r} [\mathbb{V}_{s'} [y]]. \end{aligned}$$

$$\nabla_{\theta_i} L(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s,a; \theta_i) \right) \nabla_{\theta_i} Q(s,a; \theta_i) \right].$$

The values of all the hyperparameters and optimization parameters were selected by performing an informal search on the games Pong, Breakout, Seaquest, Space Invaders and Beam Rider. We did not perform a systematic grid search owing to the high computational cost. These parameters were then held fixed across all other games. The values and descriptions of all hyperparameters are provided in Extended Data Table 1.

Hyperparameter	Value	Description
minibatch size	32	Number of training cases over which each stochastic gradient descent (SGD) update is computed.
replay memory size	1000000	SGD updates are sampled from this number of most recent frames.
agent history length	4	The number of most recent frames experienced by the agent that are given as input to the Q network.
target network update frequency	10000	The frequency (measured in the number of parameter updates) with which the target network is updated (this corresponds to the parameter C from Algorithm 1).
discount factor	0.99	Discount factor γ used in the Q-learning update.
action repeat	4	Repeat each action selected by the agent this many times. Using a value of 4 results in the agent seeing only every 4th input frame.
update frequency	4	The number of actions selected by the agent between successive SGD updates. Using a value of 4 results in the agent selecting 4 actions between each pair of successive updates.

learning rate	0.00025	The learning rate used by RMSProp.
gradient momentum	0.95	Gradient momentum used by RMSProp.
squared gradient momentum	0.95	Squared gradient (denominator) momentum used by RMSProp.
min squared gradient	0.01	Constant added to the squared gradient in the denominator of the RMSProp update.
initial exploration	1	Initial value of ϵ in ϵ -greedy exploration.
final exploration	0.1	Final value of ϵ in ϵ -greedy exploration.
final exploration frame	1000000	The number of frames over which the initial value of ϵ is linearly annealed to its final value.
replay start size	50000	A uniform random policy is run for this number of frames before learning starts and the resulting experience is used to populate the replay memory.
no-op max	30	Maximum number of “do nothing” actions to be performed by the agent at the start of an episode.

Experience Replay Buffer

- supervised learning for DNN assumes i.i.d. samples
 - independent and identically distributed samples
- a DNN learning update uses a mini-batch to compute the SGD update (SGD = stochastic gradient descent)
- mini-batch of the n most recent experience tuples is **not iid**
- solution
 - store all tuples in an experience replay buffer, then sample from that
- *prioritized experience replay*
 - train more on the data that is surprising, i.e., where the loss function is high

Video of gameplay

- <https://www.youtube.com/watch?v=W2CAghUiofY>

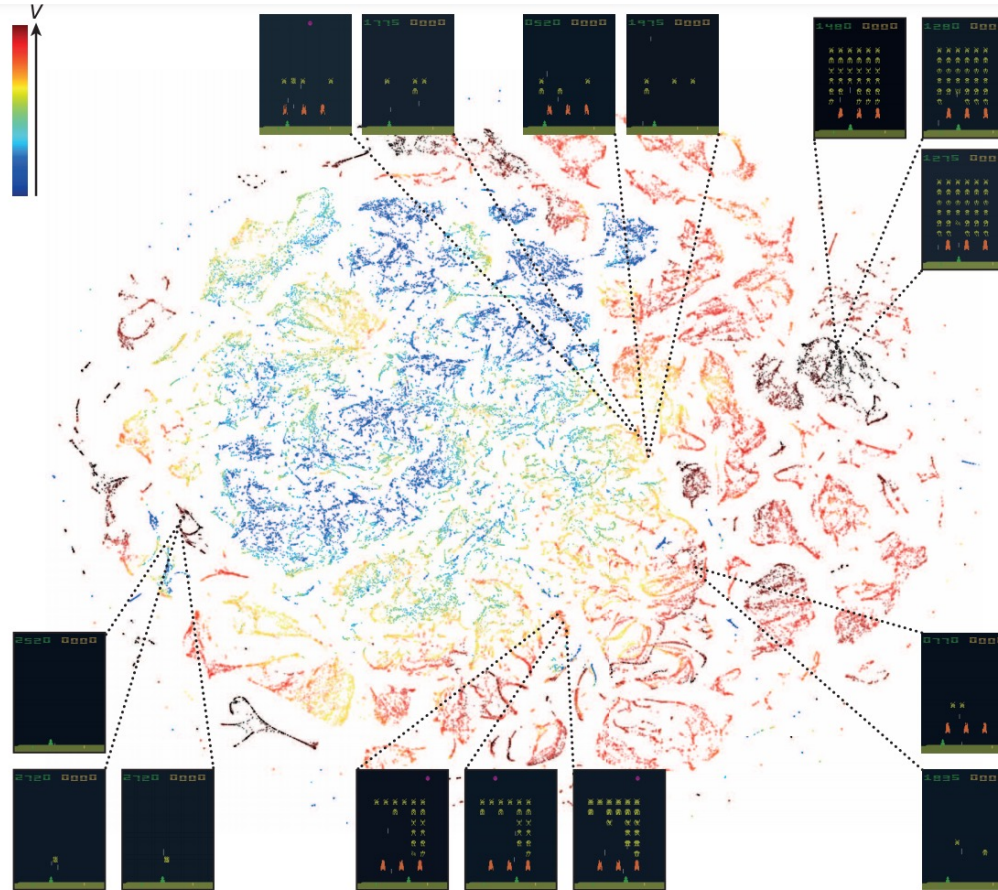


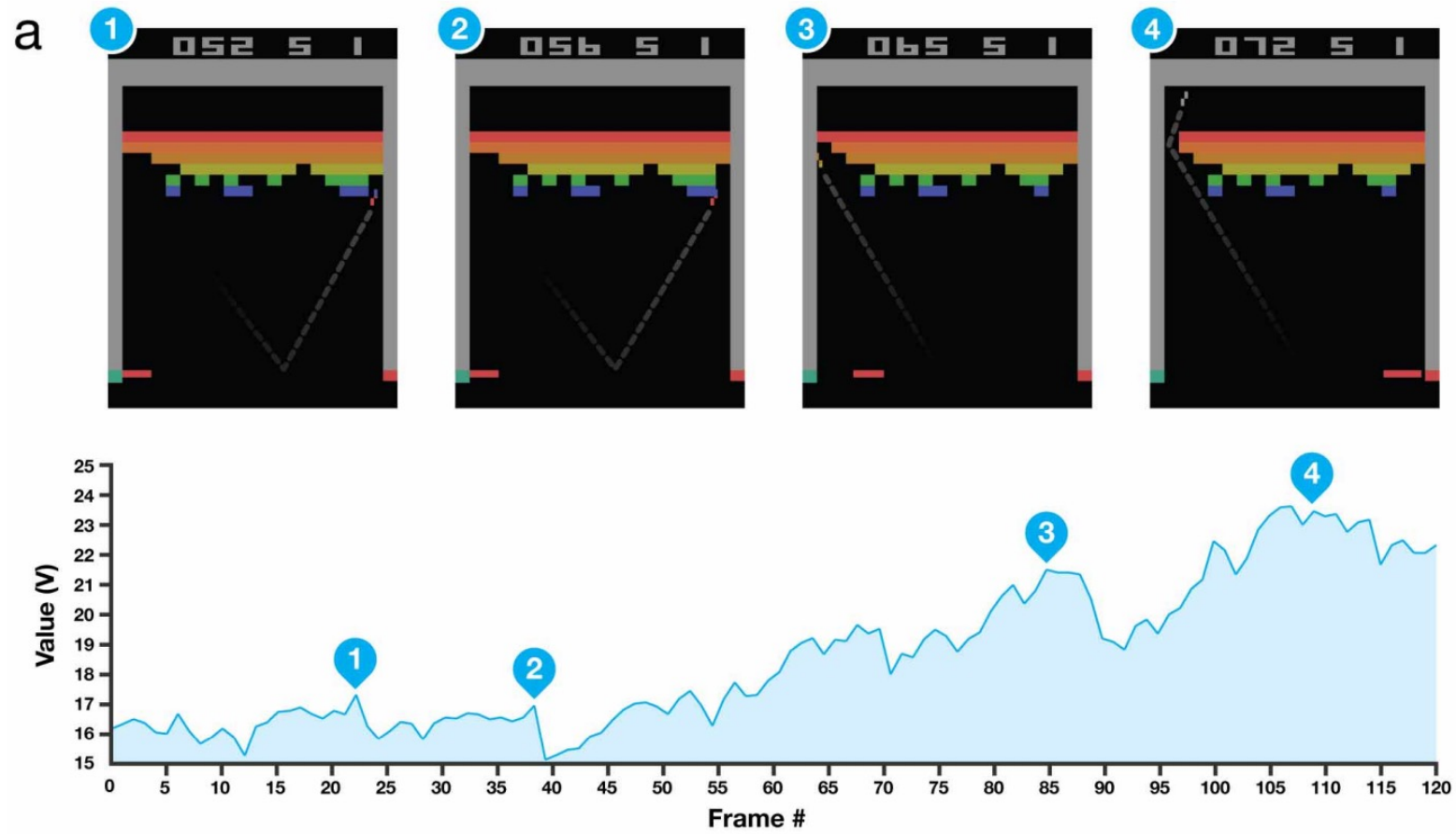
Figure 4 | Two-dimensional t-SNE embedding of the representations in the last hidden layer assigned by DQN to game states experienced while playing Space Invaders. The plot was generated by letting the DQN agent play for 2 h of real game time and running the t-SNE algorithm²⁵ on the last hidden layer representations assigned by DQN to each experienced game state. The points are coloured according to the state values (V , maximum expected reward of a state) predicted by DQN for the corresponding game states (ranging from dark red (highest V) to dark blue (lowest V)). The screenshots corresponding to a selected number of points are shown. The DQN agent

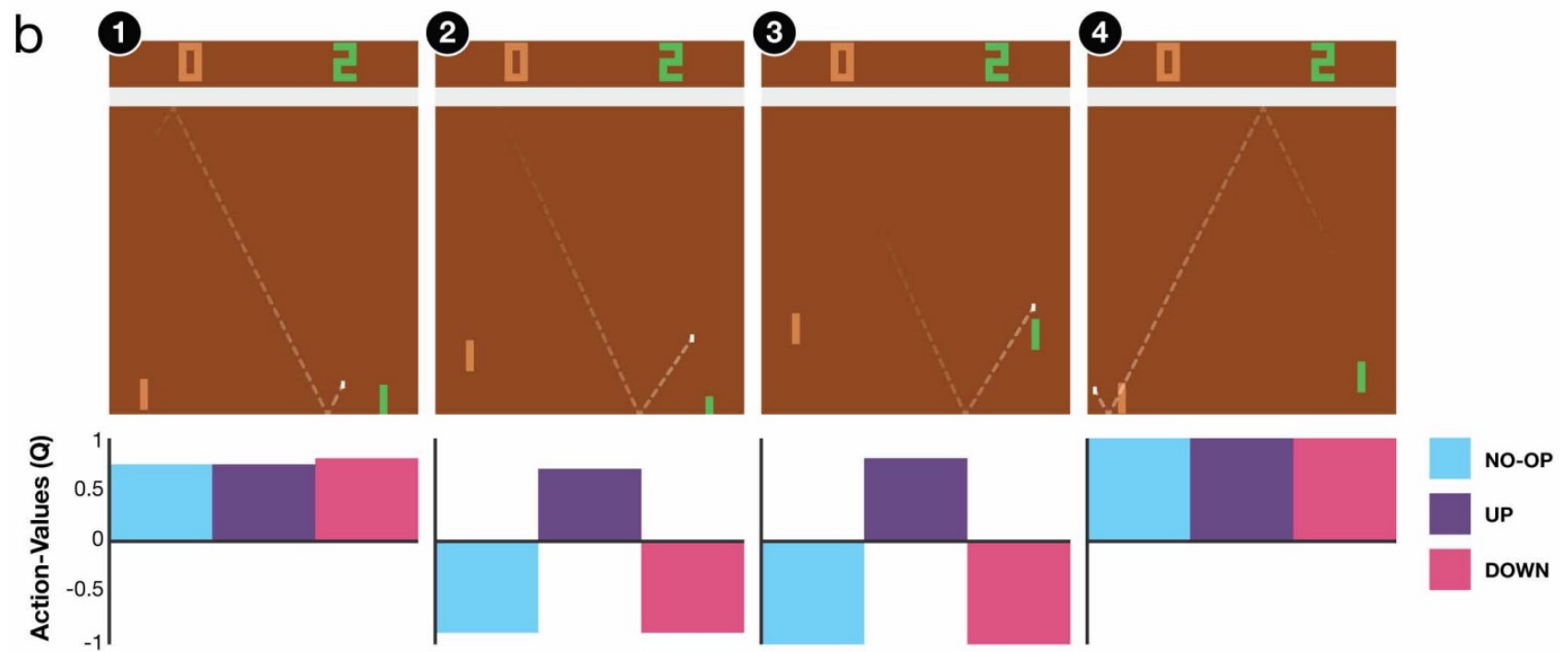
predicts high state values for both full (top right screenshots) and nearly complete screens (bottom left screenshots) because it has learned that completing a screen leads to a new screen full of enemy ships. Partially completed screens (bottom right screenshots) are assigned lower state values because less immediate reward is available. The screens shown on the bottom right and top left and middle are less perceptually similar than the other examples but are still mapped to nearby representations and similar values because the orange bunkers do not carry great significance near the end of a level. With permission from Square Enix Limited.

Interpreting the Q function



Figure 3: The leftmost plot shows the predicted value function for a 30 frame segment of the game Seaquest. The three screenshots correspond to the frames labeled by A, B, and C respectively.





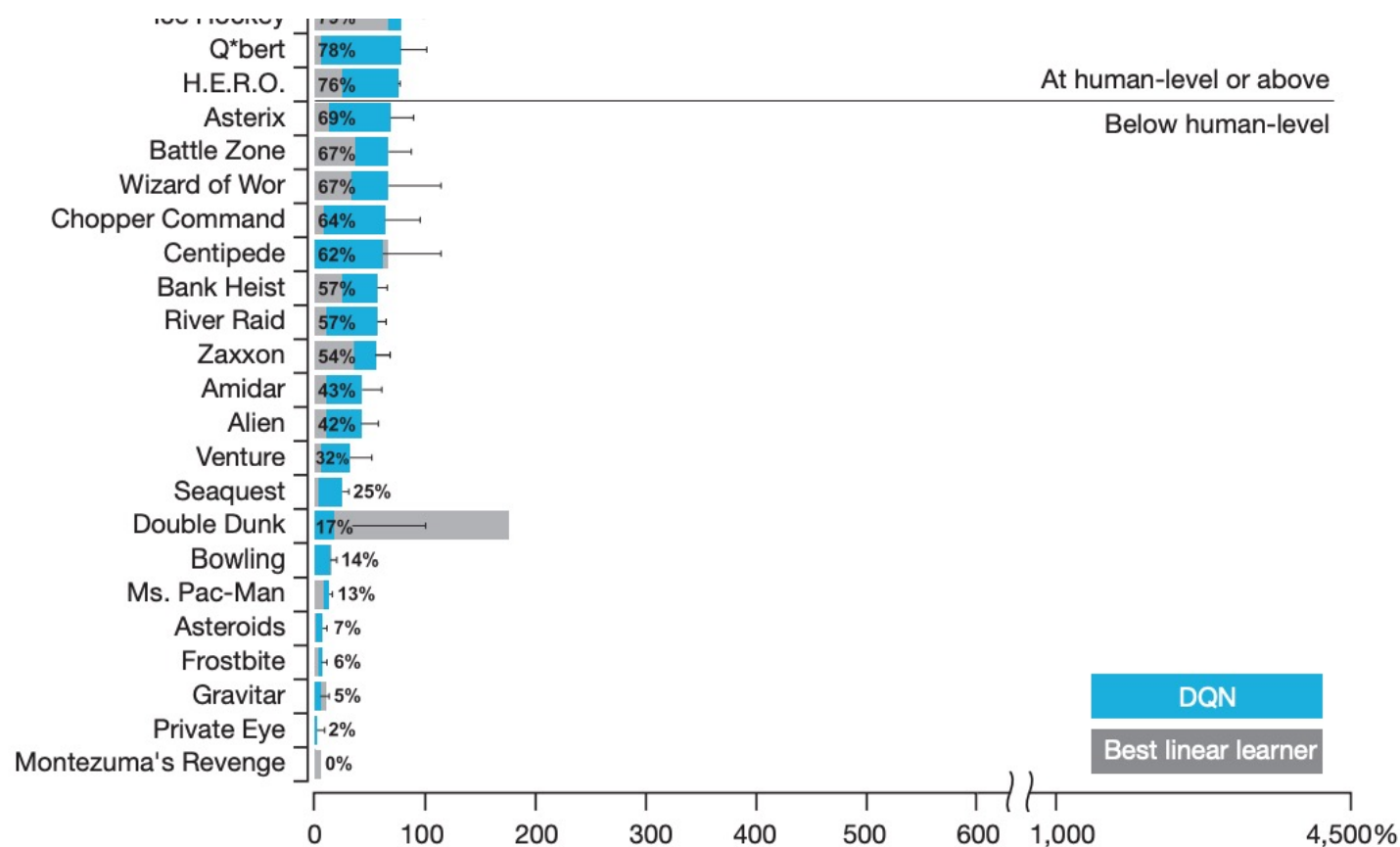
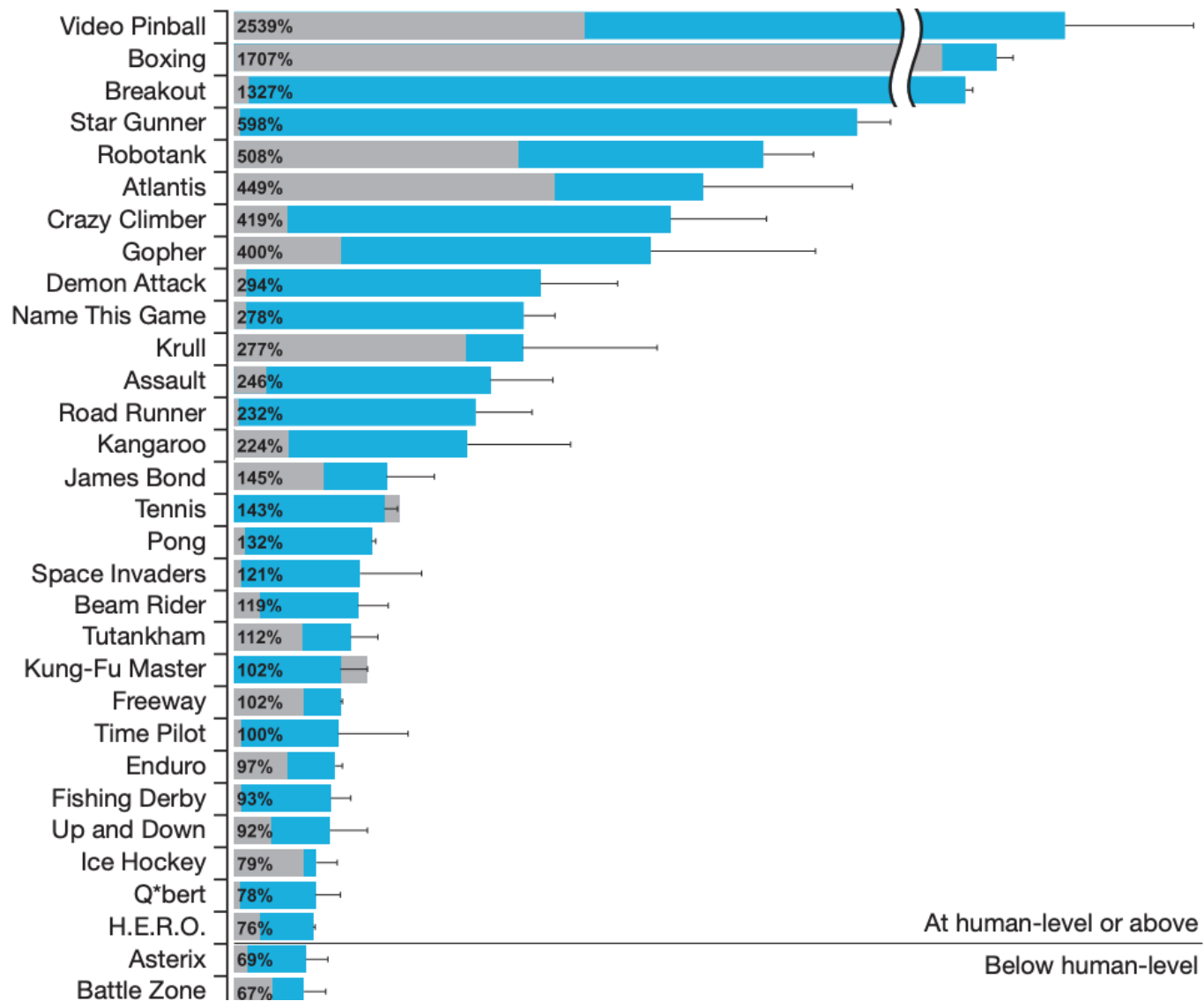


Figure 3 | Comparison of the DQN agent with the best reinforcement learning methods¹⁵ in the literature. The performance of DQN is normalized with respect to a professional human games tester (that is, 100% level) and random play (that is, 0% level). Note that the normalized performance of DQN, expressed as a percentage, is calculated as: $100 \times (\text{DQN score} - \text{random play score}) / (\text{human score} - \text{random play score})$. It can be seen that DQN

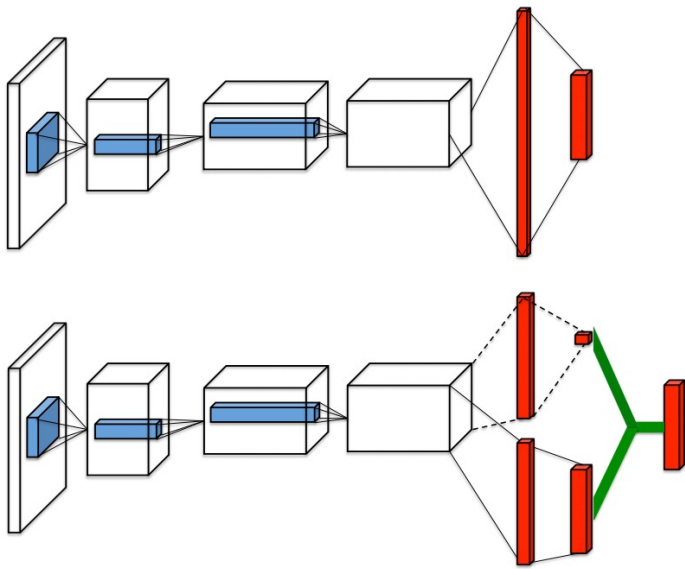
outperforms competing methods (also see Extended Data Table 2) in almost all the games, and performs at a level that is broadly comparable with or superior to a professional human games tester (that is, operationalized as a level of 75% or above) in the majority of games. Audio output was disabled for both human players and agents. Error bars indicate s.d. across the 30 evaluation episodes, starting with different initial conditions.



Dueling DQN

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad \mathbb{E}_{a \sim \pi(s)} [A^\pi(s, a)] = 0.$$

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha)$$

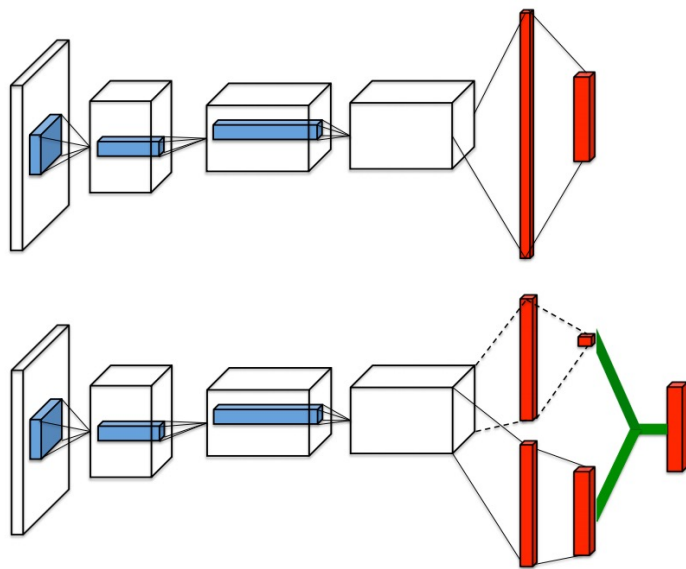


$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \max_{a' \in |\mathcal{A}|} A(s, a'; \theta, \alpha) \right)$$

or

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right)$$

[Wang et al. 2015]



Distributional RL

$$Q(x, a) = \mathbb{E} R(x, a) + \gamma \mathbb{E} Q(X', A')$$



$$Z(x, a) \stackrel{D}{=} R(x, a) + \gamma Z(X', A')$$

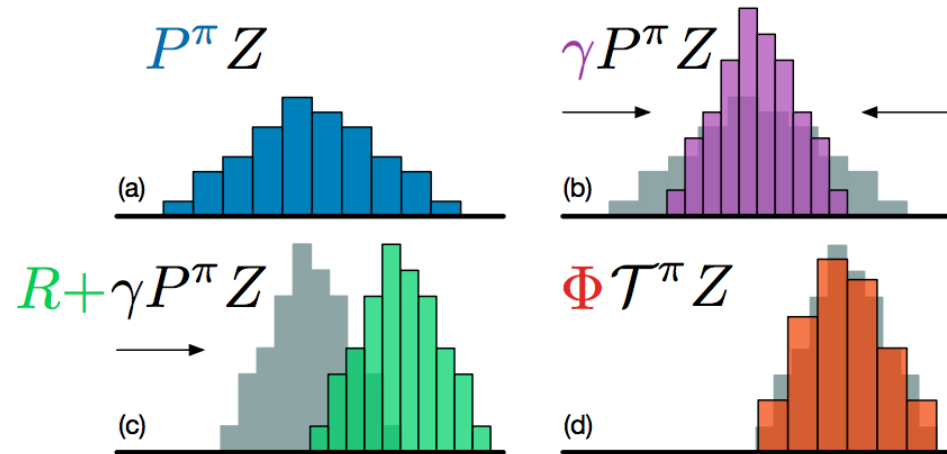
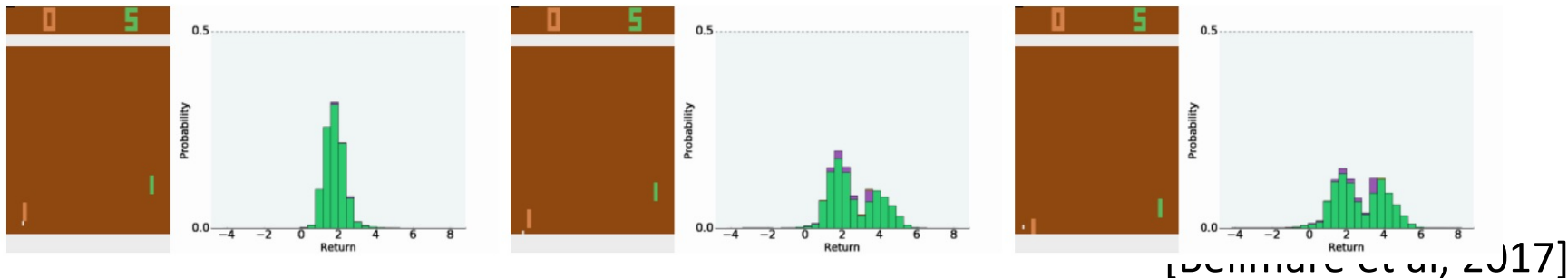
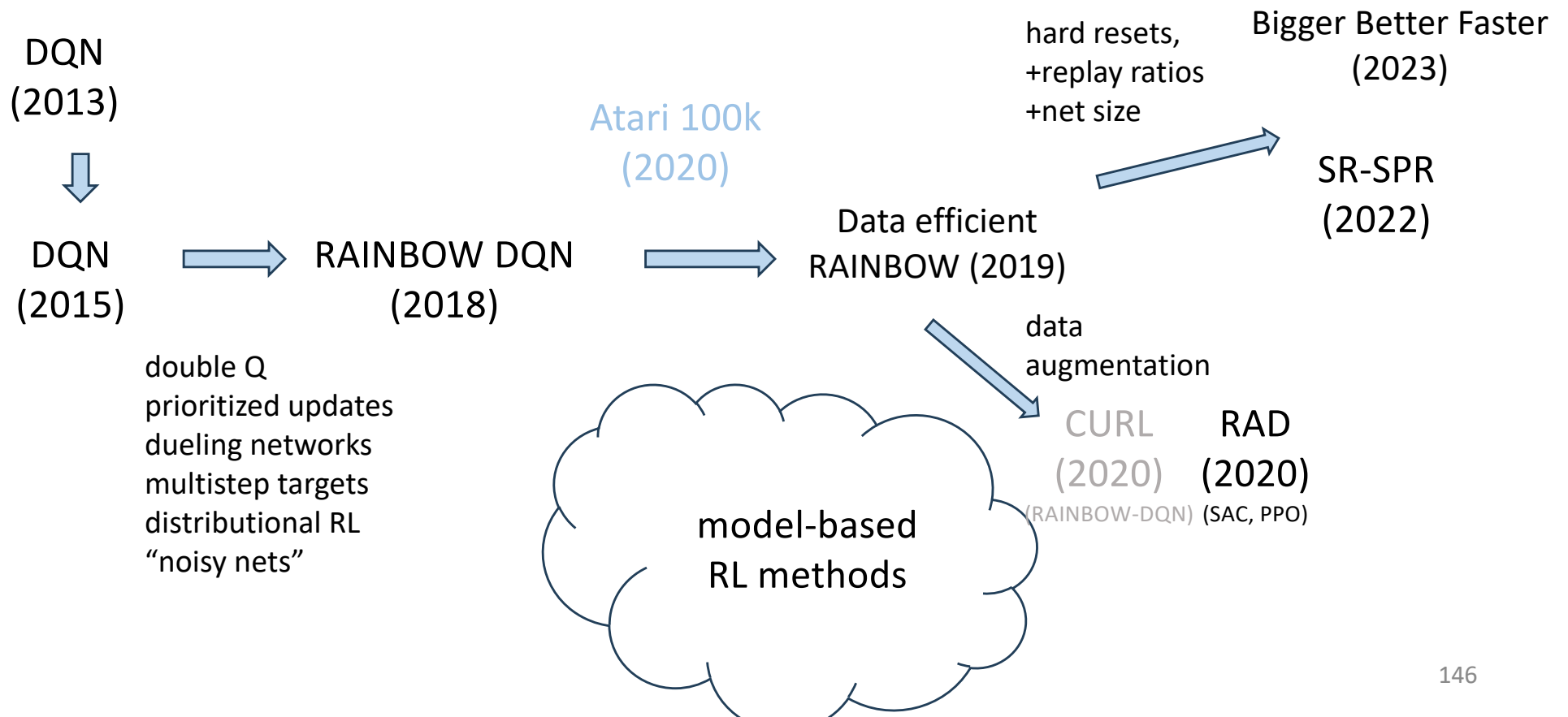


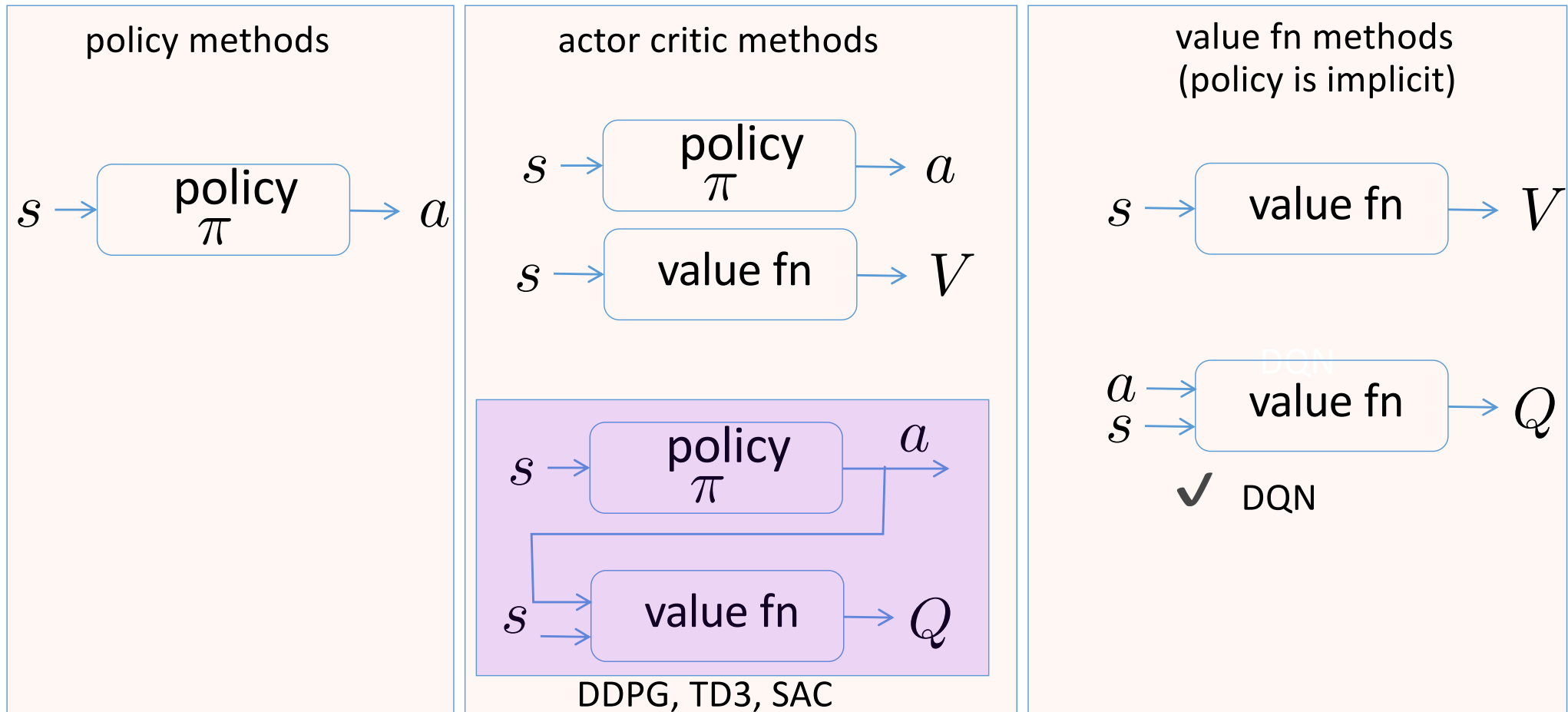
Figure 1. A distributional Bellman operator with a deterministic reward function: (a) Next state distribution under policy π , (b) Discounting shrinks the distribution towards 0, (c) The reward shifts it, and (d) Projection step (Section 4).



DQN evolution (on Atari)



Deep RL: key building blocks

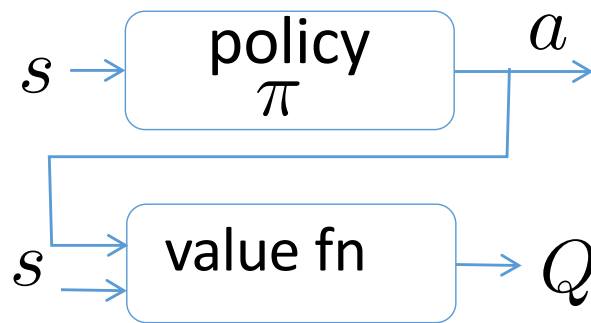


DDPG: Deep Deterministic Policy Gradients (off policy algorithm; 2014-2016)

- Can DQN be extended to problems with continuous actions?

No: $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

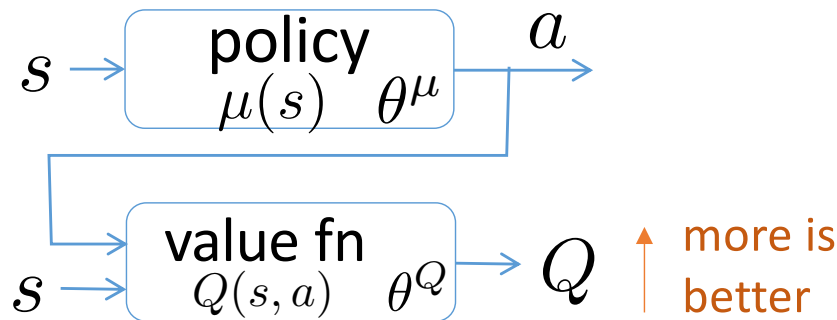
?



In continuous action spaces, greedy policy improvement becomes problematic, requiring a global maximisation at every step. Instead, a simple and computationally attractive alternative is to move the policy in the direction of the gradient of Q , rather than globally maximising Q .

DDPG

Q-learning for continuous actions, off-policy



key to updating the policy – chain rule:

$$\frac{\partial Q}{\partial \theta^\mu} = \frac{\partial Q}{\partial a} \frac{\partial a}{\partial \theta^\mu}$$

The DPG algorithm maintains a parameterized actor function $\mu(s|\theta^\mu)$ which specifies the current policy by deterministically mapping states to a specific action. The critic $Q(s, a)$ is learned using the Bellman equation as in Q-learning. The actor is updated by following the applying the chain rule to the expected return from the start distribution J with respect to the actor parameters:

policy gradient

$$\begin{aligned} \nabla_{\theta^\mu} J &\approx \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_{\theta^\mu} Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t|\theta^\mu)}] && \text{[Lillicrap et al. 2016]} \\ &= \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_a Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s=s_t}] && (6) \end{aligned}$$

J : performance objective,
i.e. expected returns from visited states

trajectories sampled from (off) policy β

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R **experience replay buffer; old transitions remain valid**

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for t = 1, T **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and **exploration noise**

Ornstein-Uhlenbeck process
(temporally coherent noise)

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$ **use the future action from (target) policy)**

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$ **update Q(s,a) network**

 Update the actor policy using the sampled gradient:

$$\nabla_{\theta^\mu} \mu|_{s_i} \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i} \quad \text{update policy network}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

Q' tracks Q, with a lag

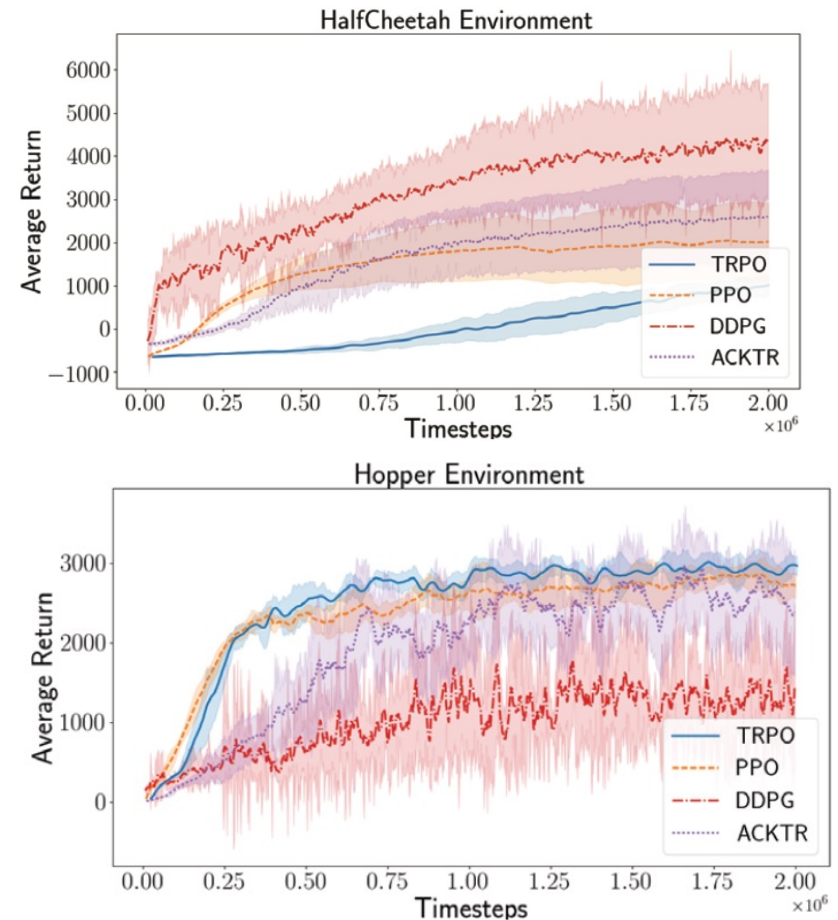
pi' tracks pi, with a lag

exponential averaging /
Polyak averaging /
basic autoregressive filter

end for
end for

Issues: stability, exploration

Results As shown in Figure 4, in environments with stable dynamics (e.g. HalfCheetah-v1), DDPG outperforms all other algorithms. However, as dynamics become more unstable (e.g. in Hopper-v1) performance gains rapidly diminish. As DDPG is an off-policy method, exploration noise can cause sudden failures in unstable environments. Therefore, learning a proper Q -value estimation of expected returns is difficult, particularly since many exploratory paths will result in failure. Since failures in such tasks are characterized by shortened trajectories, a local optimum in this case would be simply to survive until the maximum length of the trajectory (corresponding to one thousand timesteps and similar reward due to a survival bonus in the case of Hopper-v1). As can be seen in Figure 4, DDPG with Hopper does exactly this. This is a clear example where showing only the favourable and stable HalfCheetah when reporting DDPG-based experiments would be unfair. [Deep RL that matters, AAAI 2018]



Is DDPG really “Off-Policy”?

“Off-Policy Deep Reinforcement Learning without Exploration” (2019)

- “we demonstrate that due to errors introduced by extrapolation, standard offpolicy deep reinforcement learning algorithms, such as DQN and DDPG, are incapable of learning without data correlated to the distribution under the current policy, making them ineffective for this fixed batch setting.”

TD3: Twin Delayed DDPG

Commonly used! Big improvement over DDPG

Addressing Function Approximation Error in Actor-Critic Methods

Scott Fujimoto¹ Herke van Hoof² David Meger¹

[ICML 2018]

- three fixes to DDPG:
 - clipped double Q learning (“twin” Q)
 - delayed policy updates (as compared to Q fn)
 - target policy smoothing: adding noise to Q fn targets
- simpler to implement than SAC

Overestimation bias:
need for “twin”

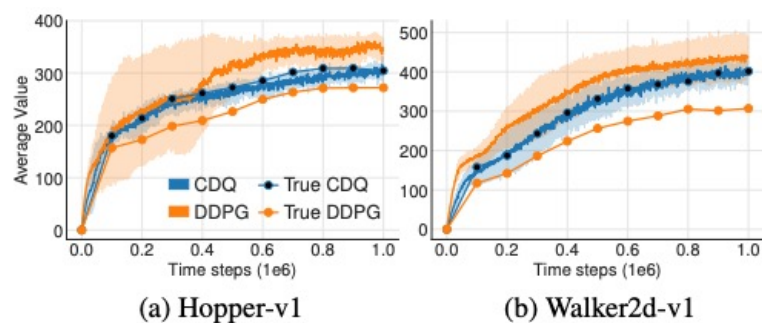


Figure 1. Measuring overestimation bias in the value estimates of DDPG and our proposed method, Clipped Double Q-learning (CDQ), on MuJoCo environments over 1 million time steps.

Target value fluctuations:
need for “delayed”

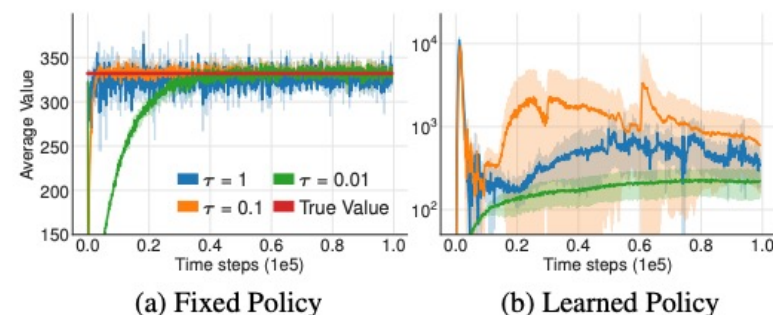


Figure 3. Average estimated value of a randomly selected state on Hopper-v1 without target networks, ($\tau = 1$), and with slow-updating target networks, ($\tau = 0.1, 0.01$), with a fixed and a learned policy.

Algorithm 1 TD3

Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ

Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$

Initialize replay buffer $\mathcal{B}$

for $t = 1$ **to** T **do**

 Select action with exploration noise $a \sim \pi(s) + \epsilon$,

$\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'

 Store transition tuple (s, a, r, s') in $\mathcal{B}$

 Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$

$\tilde{a} \leftarrow \pi_{\phi'}(s) + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$

$y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$

 Update critics $\theta_i \leftarrow \min_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$

if $t \bmod d$ **then**

 Update ϕ by the deterministic policy gradient:

$\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$

 Update target networks:

$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$

$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$

end if

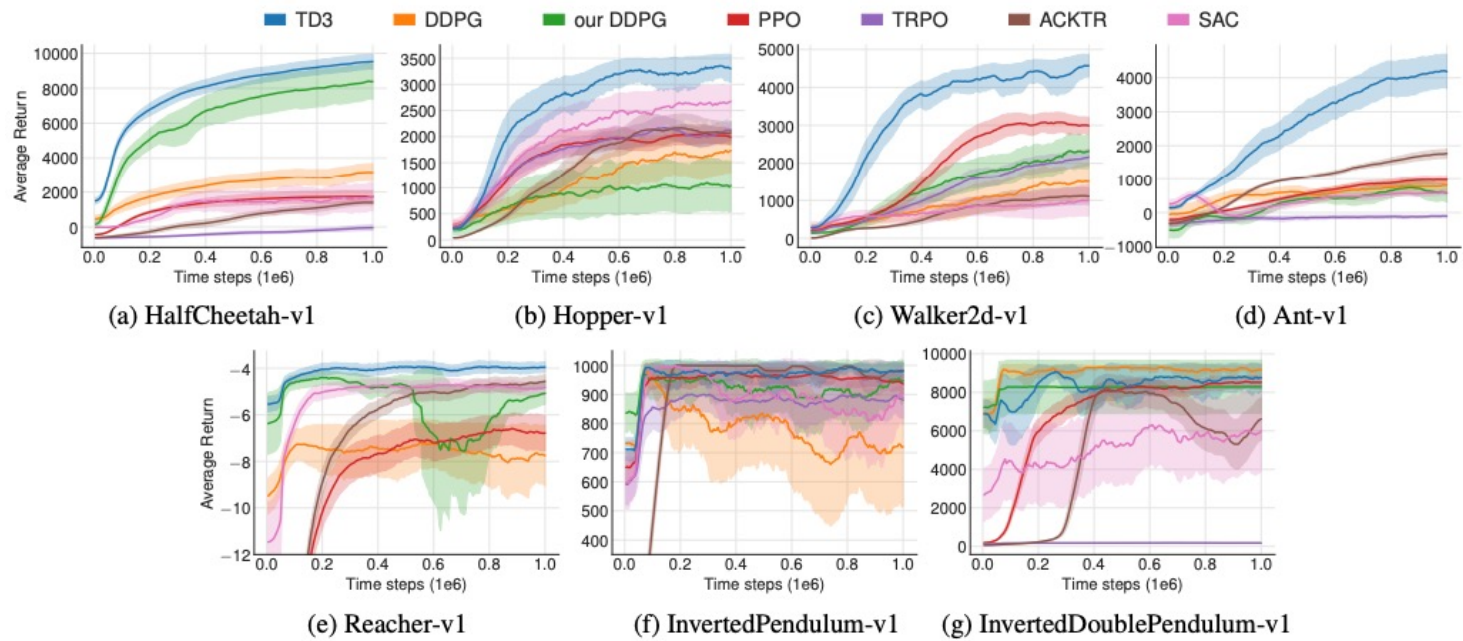
end for

smooth & clip
twin

delayed

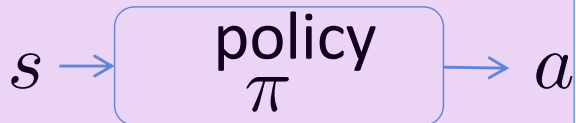
(recent versions of
SAC now also clip and
uses twin Q fns)

TD3 Results



Policy-based Methods

policy methods



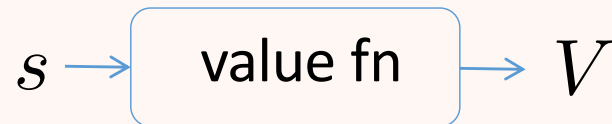
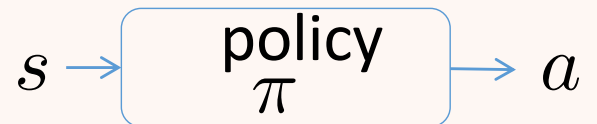
Policy Search:

- CMA, CES

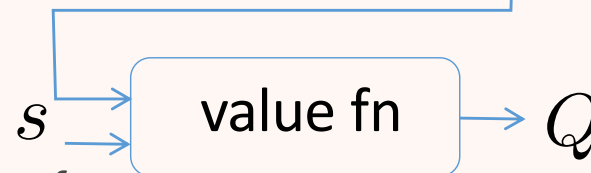
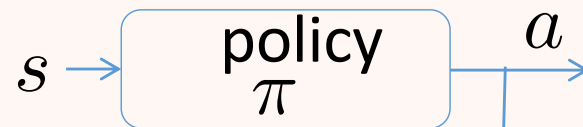
Policy Gradient:

- REINFORCE, A2C, PPO

actor critic methods

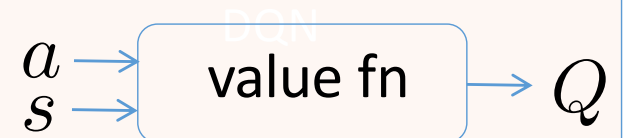
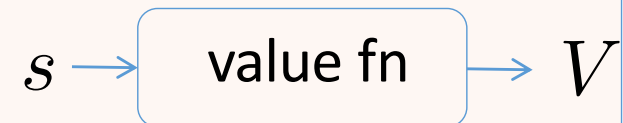


Policy Gradient



✓ DDPG, TD3

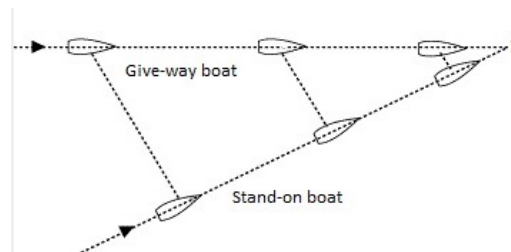
value fn methods (policy is implicit)



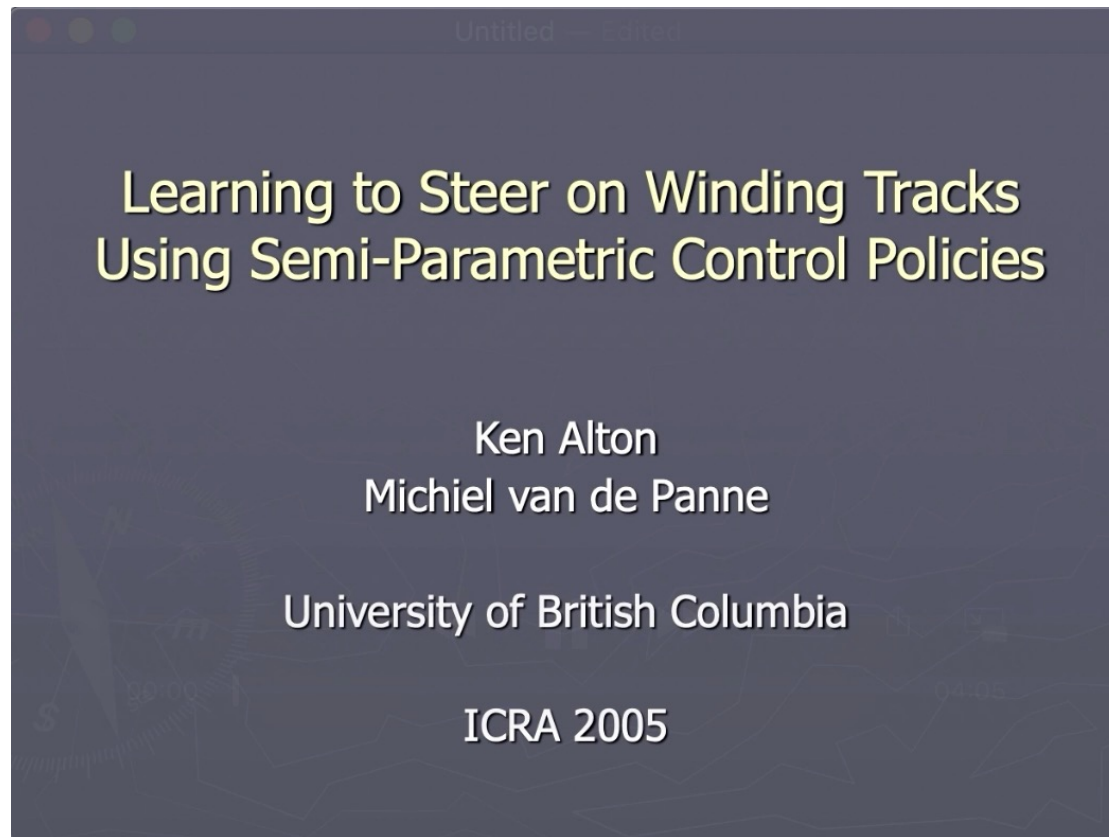
✓ DQN, DQN++

Why policy-based methods?

- to handle continuous actions
- good solutions can (sometimes) be described using simple policies, e.g., "constant bearing" strategy

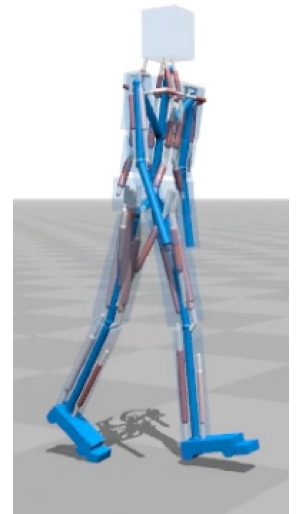
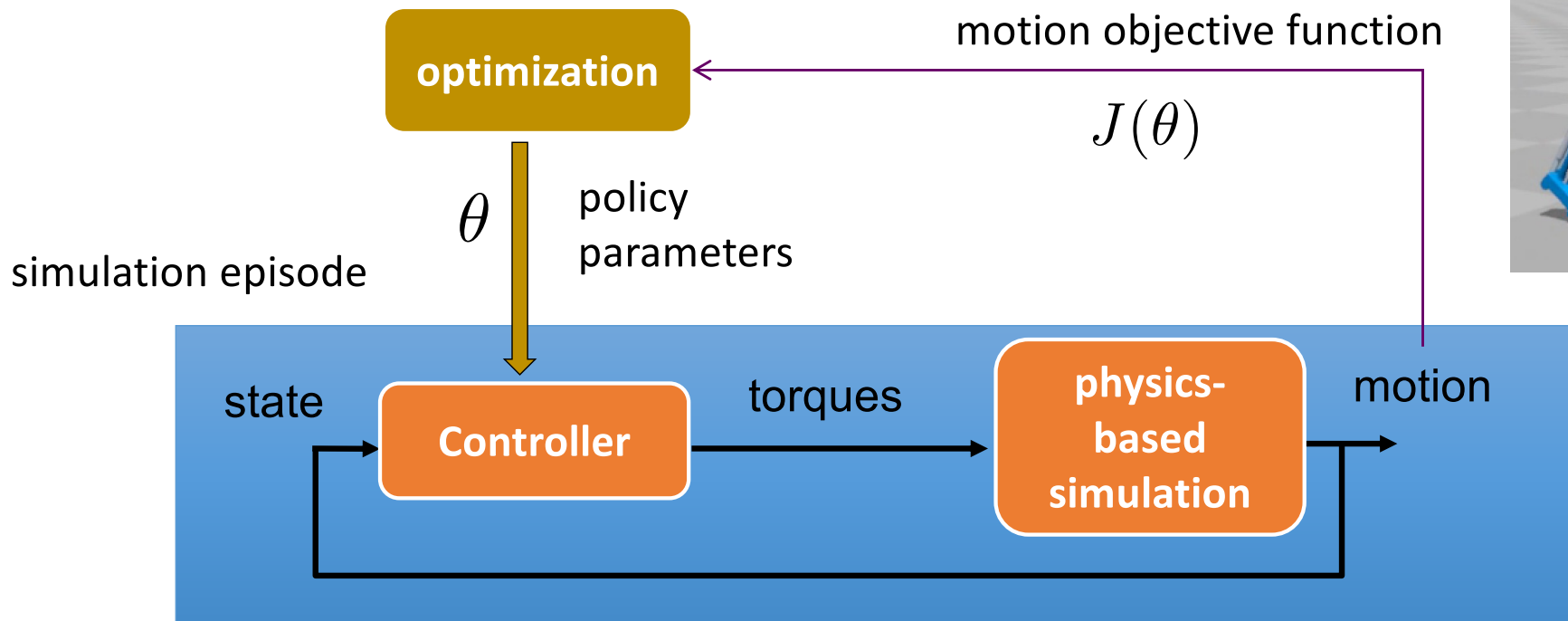


Learning to Steer (forward and backward)



Policy Search / Hill Climbing / Black Box / Evolutionary / Gradient-Free / Derivative-Free Optimization / Parameter-Exploring Policy Gradients

Optimize $J(\theta)$ using **only** total episode returns



Policy optimization problem

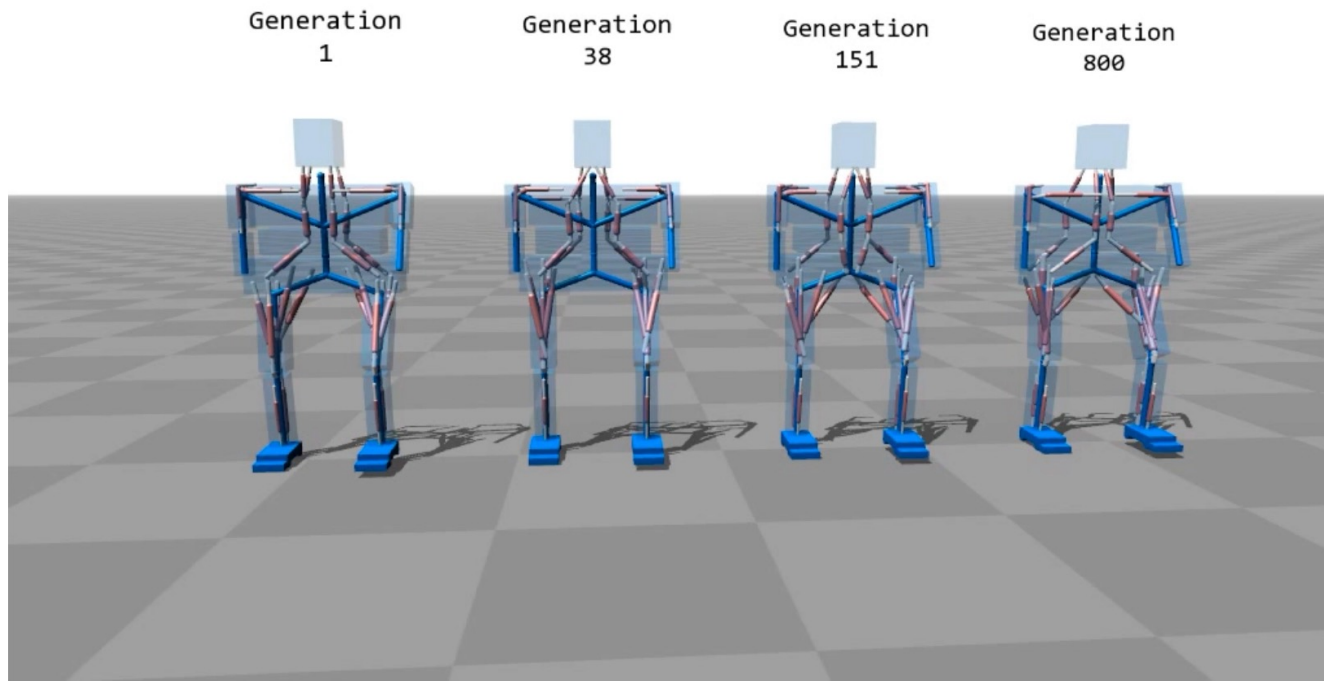
policy parameters:

Subject	Parameters
Muscle physiology	3–30 *
Muscle geometry	12–39 *
State transition	3
Target features	14
Feedback control	14–63 *
Initial character state	6

objective:

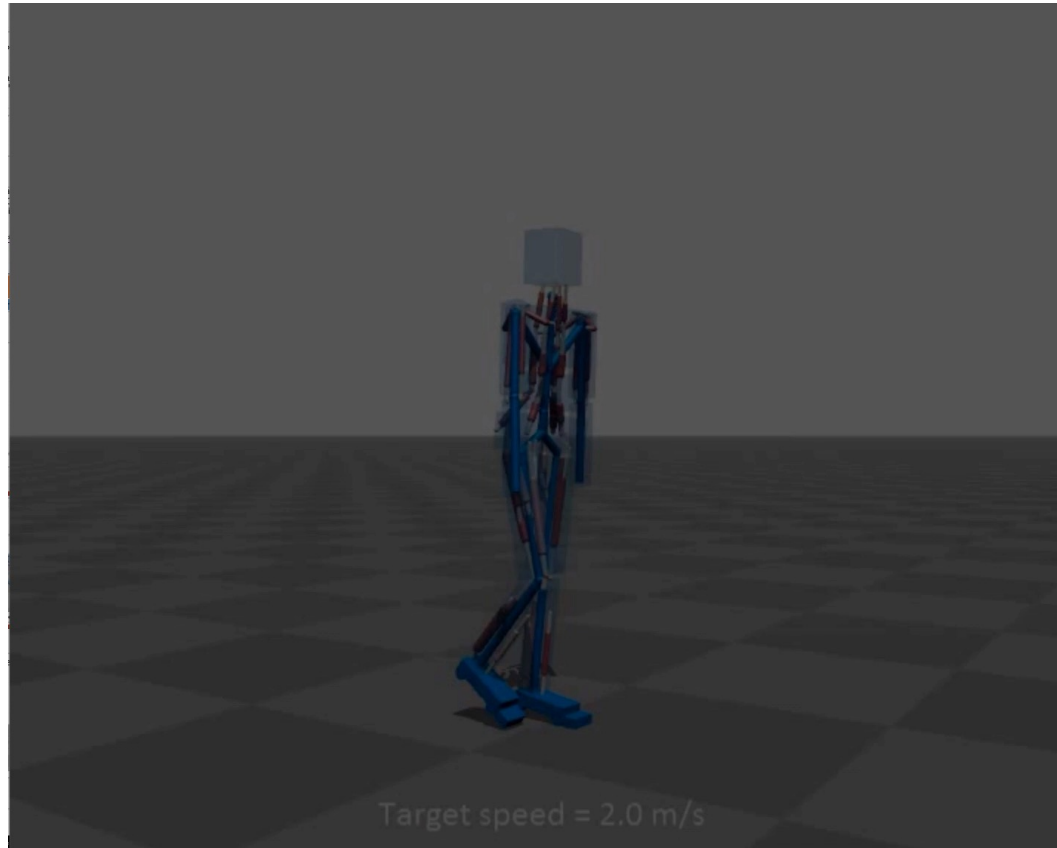
$$\bar{E}(\mathbf{K}) = \bar{E}_{\text{speed}} + \bar{E}_{\text{headori}} + \bar{E}_{\text{headvel}} + \bar{E}_{\text{slide}} + \bar{E}_{\text{effort}}$$

Biomechanical Locomotion of Imaginary Creatures

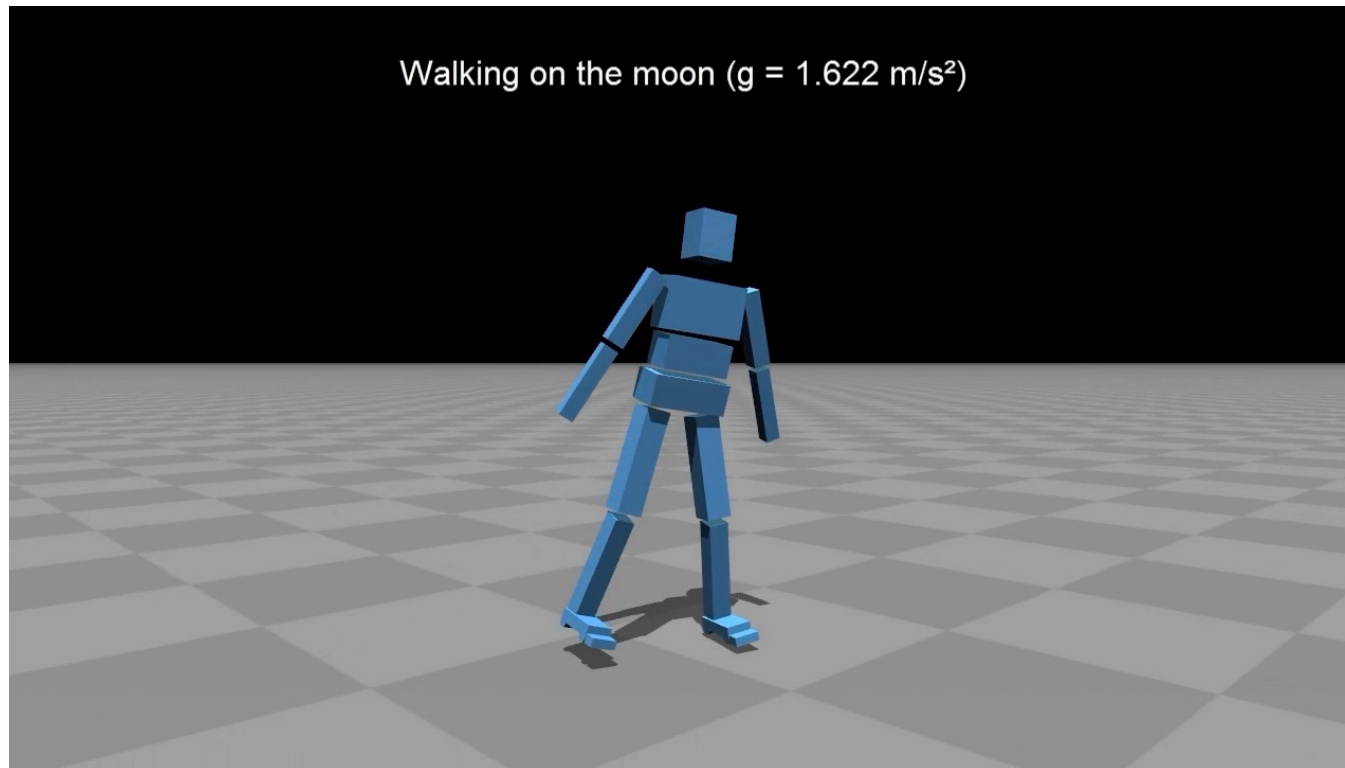


[Flexible Muscle-based Locomotion for Bipedal Creatures, SIGGRAPH ASIA 2013]

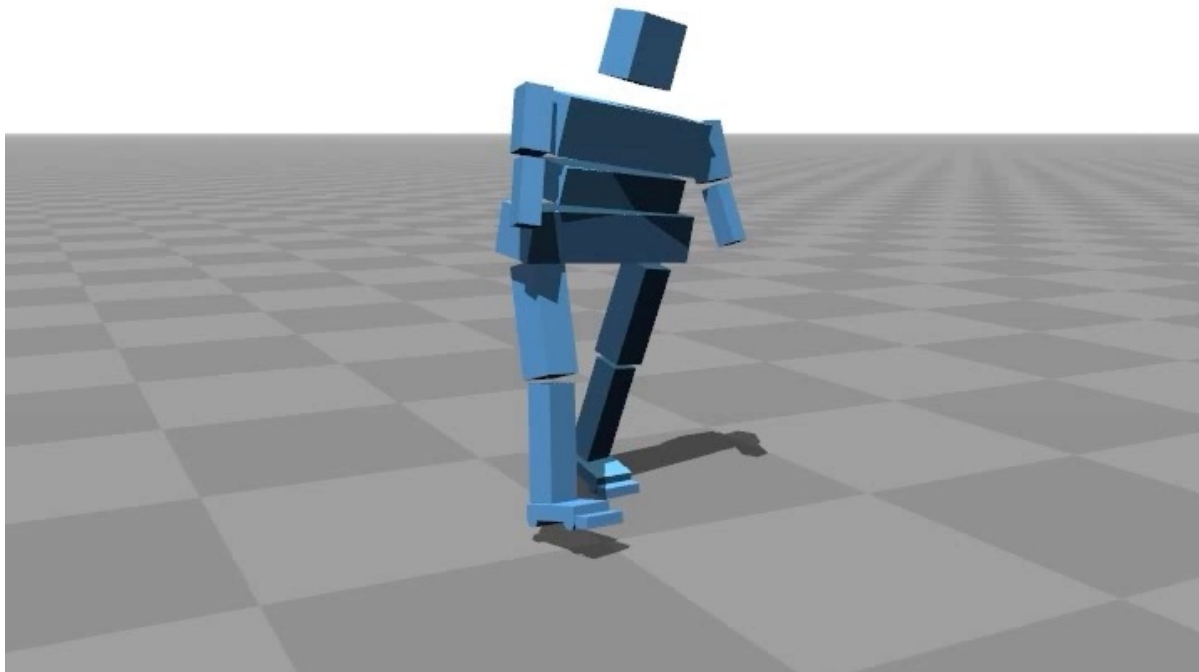
Optimization for Various Speeds



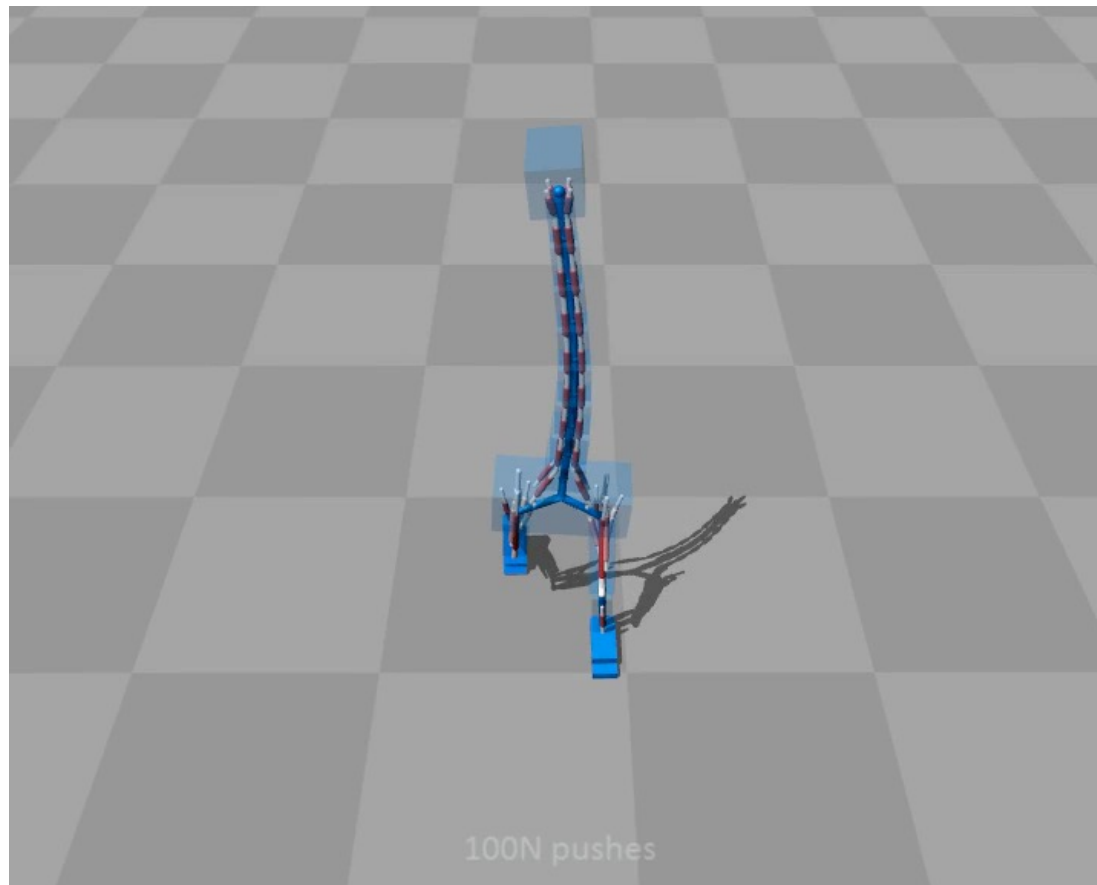
Moon Walk



Robustness



Codesign of Body and Control



Example Gradient-Free Methods:

CEM: Cross Entropy Method

CMA: Covariance Matrix Adaptation

Cross-Entropy Method

$$\max_{\theta} U(\theta) = \max_{\theta} \mathbb{E}\left[\sum_{t=0}^H R(s_t) | \pi_{\theta}\right]$$

- Views U as a black box
- Ignores all other information other than U collected during episode

= evolutionary algorithm

population: $P_{\mu^{(i)}}(\theta)$

CEM:

```
for iter i = 1, 2, ...  
  for population member e = 1, 2, ...  
    sample  $\theta^{(e)} \sim P_{\mu^{(i)}}(\theta)$   
    execute roll-outs under  $\pi_{\theta^{(e)}}$   
    store  $(\theta^{(e)}, U(e))$   
  endfor  
   $\mu^{(i+1)} = \arg \max_{\mu} \sum_{\bar{e}} \log P_{\mu}(\theta^{(\bar{e})})$   
  where  $\bar{e}$  indexes over top p %  
endfor
```

[Abbeel]¹⁷⁰

Cross-Entropy Method

■ Can work embarrassingly well

Method	Mean Score	Reference
Nonreinforcement learning		
Hand-coded	631,167	Dellacherie (Fahey, 2003)
Genetic algorithm	586,103	(Böhm et al., 2004)
Reinforcement learning		
Relational reinforcement learning+kernel-based regression	≈50	Ramon and Driessens (2004)
Policy iteration	3183	Bertsekas and Tsitsiklis (1996)
Least squares policy iteration	<3000	Lagoudakis, Parr, and Littman (2002)
Linear programming + Bootstrap	4274	Farias and van Roy (2006)
Natural policy gradient	≈6800	Kakade (2001)
CE+RL	21,252	
CE+RL, constant noise	72,705	
CE+RL, decreasing noise	348,895	

István Szita and András Lörincz. "Learning Tetris using the noisy cross-entropy method". In: *Neural computation* 18.12 (2006), pp. 2936–2941

Approximate Dynamic Programming Finally Performs Well in the Game of Tetris

[NIPS 2013]

Victor Gabillon
INRIA Lille - Nord Europe,
Team SequeL, FRANCE
victor.gabillon@inria.fr

Mohammad Ghavamzadeh*
INRIA Lille - Team SequeL
& Adobe Research
mohammad.ghavamzadeh@inria.fr

Bruno Scherrer
INRIA Nancy - Grand Est,
Team Maia, FRANCE
bruno.scherrer@inria.fr

John Schulman & Pieter Abbeel – OpenAI + UC Berkeley

Closely Related Approaches

CEM:

```

for iter i = 1, 2, ...
  for population member e = 1, 2, ...
    sample  $\theta^{(e)} \sim P_{\mu^{(i)}}(\theta)$ 
    execute roll-outs under  $\pi_{\theta^{(e)}}$ 
    store  $(\theta^{(e)}, U(e))$ 
  endfor
   $\mu^{(i+1)} = \arg \max_{\mu} \sum_{\bar{e}} \log P_{\mu}(\theta^{(\bar{e})})$ 
  where  $\bar{e}$  indexes over top p %
endfor
    
```

■ Reward Weighted Regression (RWR)

- Dayan & Hinton, NC 1997; Peters & Schaal, ICML 2007

$$\mu^{(i+1)} = \arg \max_{\mu} \sum_e q(U(e), P_{\mu}(\theta^{(e)})) \log P_{\mu}(\theta^{(e)})$$

■ Policy Improvement with Path Integrals (PI²)

- PI²: Theodorou, Buchli, Schaal JMLR2010; Kappen, 2007; (PI²-CMA: Stulp & Sigaud ICML2012)

$$\mu^{(i+1)} = \arg \max_{\mu} \sum_e \exp(\lambda U(e)) \log P_{\mu}(\theta^{(e)})$$

■ Covariance Matrix Adaptation Evolutionary Strategy (CMA-ES)

- CMA: Hansen & Ostermeier 1996; (CMA-ES: Hansen, Muller, Koumoutsakos 2003)

$$(\mu^{(i+1)}, \Sigma^{(i+1)}) = \arg \max_{\mu, \Sigma} \sum_{\bar{e}} w(U(\bar{e})) \log \mathcal{N}(\theta^{(\bar{e})}; \mu, \Sigma)$$

■ PoWER

- Kober & Peters, NIPS 2007 (also applies importance sampling for sample re-use)

$$\mu^{(i+1)} = \mu^{(i)} + \left(\sum_e (\theta^{(e)} - \mu^{(i)}) U(e) \right) / \left(\sum_e U(e) \right)$$

[Abbeel]

Cross-Entropy / Evolutionary Methods

- Full episode evaluation, parameter perturbation
- Simple
- Main caveat: best when intrinsic dimensionality not too high
 - i.e., number of population members comparable to or larger than number of (effective) parameters
 - in practice OK if low-dimensional θ and willing to do many runs
 - Easy-to-implement baseline, great for comparisons!

[Abbeel]

Considerations

■ Pros:

- Work with arbitrary parameterization, even non-differentiable
- Embarrassingly easy to parallelize

■ Cons:

- Not very sample efficient since ignores temporal structure

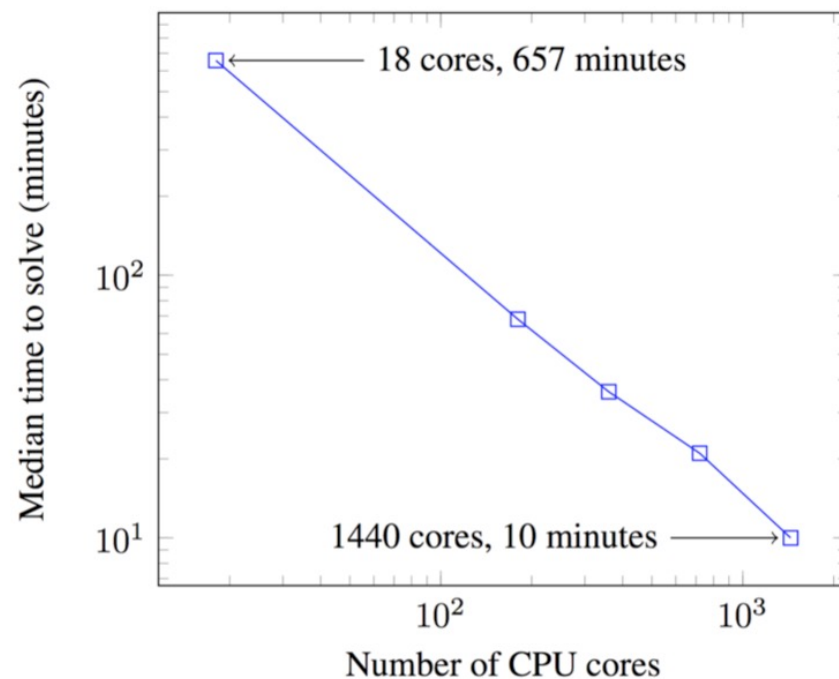


Figure 1. Time to reach a score of 6000 on 3D Humanoid with different number of CPU cores. Experiments are repeated 7 times and median time is reported.

[Salimans, Ho, Chen, Sutskever, 2017]